

Tools in the Context Window: An Open Benchmark for MCP versus CLI Tool Use Across Agent Scaffoldings

Draft — results pending. Figures and every number marked `<TBD>` are filled from `results/runs.jsonl` by `bench/analyze.py`; nothing in this file is hand-entered.

Abstract

An agent needs a way to act on the world, and there are two dominant ways to give it one. The Model Context Protocol publishes a catalogue of typed tools that the host injects into the model’s context. A command-line tool publishes nothing: the agent writes a command string and reads what comes back. Practitioners report that the first approach is far more expensive in tokens than the second, with figures ranging from a modest multiple to more than an order of magnitude, but these reports are single-setup anecdotes and none of them can be re-run.

We present an open benchmark that measures the difference on one fixed task across four agent scaffoldings — Claude Code, OpenAI Codex, qwen-code and pi — and nine models spanning hosted frontier systems and locally-served open weights. Each scaffolding runs the same workflow twice, once with a GitHub MCP server attached and once with its native shell and the `gh` command-line tool, in its default configuration. We record initial context size, total and cached tokens, the number of tool calls and which tools were called, tool-call success, the percentage of the task completed against a rubric, and a theoretical cost computed from one public price list applied uniformly to every cell.

`<TBD: headline result.>`

The harness, the pinned fixture repository, the task, the scoring rubric and the raw run logs are public, so every number here can be re-derived and every claim can be attacked with the same instrument that produced it.

1. Introduction

The question this paper measures is narrow and practical: when you give an agent a capability, what does the delivery mechanism cost you?

Two mechanisms dominate. Under the Model Context Protocol, a server advertises a catalogue of tools with names, descriptions and typed parameter schemas, and the host places that catalogue in the model’s context so the model can select from it. Under the command-line approach, the agent already has a shell, and a capability arrives as knowledge of which commands to run — from the model’s training, from a short skill document, or from a `--help` invocation the agent issues itself.

The mechanisms differ in where the description of the capability lives. MCP puts it in the context window, priced per token, on every request of the session. A CLI puts it in the filesystem, priced at nothing, and the agent pays only for the command it writes and the output it reads back.

That asymmetry is widely asserted. A matched-task comparison reports roughly 35× more tokens through MCP than through a CLI [MindStudio 2026]. A single GitHub query is measured at ~1,365 tokens via `gh` against ~44,026 via the corresponding MCP server [Vensas 2026]. A GitHub MCP server exposing 93 tools is reported to add ~55,000 tokens of registry overhead at session start [Reinhard 2026]. Practitioner write-ups put smaller catalogues at 13,700 and 18,000 tokens and observe that “7–9% of context is gone before you start” [Zechner 2026].

These numbers disagree with each other by more than an order of magnitude, and the disagreement is not surprising: each was measured on one scaffolding, with one model, on one task, and none ships a harness. A practitioner deciding how to build an agent cannot tell from this literature whether the penalty is 1.2× or 35×, nor what it depends on.

We think the disagreement is itself the finding, and that it is explained by variables the reports do not hold fixed. This paper’s contribution is therefore not a single ratio. It is **an instrument**: a task, a rubric, a set of adapters that normalise four incompatible telemetry formats, and a published set of runs against which any of our claims can be checked.

Contributions

1. **An open, re-runnable benchmark** for MCP-versus-CLI tool use, spanning four scaffoldings and nine models, with a pinned fixture so ground truth cannot drift.
2. **A measurement across scaffoldings rather than within one**, which shows that the choice of scaffolding moves the result by more than the choice of surface.
3. **A set of measurement hazards**, each found by producing a wrong answer first, that any replication must control or silently mis-measure (§4).
4. **Raw run logs and a tool register** — not just how many tools were called, but which — so the mechanism behind the token difference is inspectable.

2. Related work

⟨TBD: expand — practitioner benchmarks, agent-harness papers, tool-selection work, context-management literature.⟩

3. Method

3.1 The task

One workflow, run against a frozen snapshot of a real teaching repository (Lamb-Project/aawd-e1-fixture, tag fixture-v1). The agent is asked five questions and must answer all five in one numbered list:

1. the repository’s default branch;
2. the top-level directory names;
3. the first heading of README.md;
4. the git tags that exist;
5. how many files are in the week-01 directory tree.

Each item is checkable independently against ground truth, so a run is scored as a **percentage of the task completed** rather than pass or fail. Weak models are expected to fail partially, and where they fail is data.

Items 1–4 are lookups of increasing depth. **Item 5 is the discriminating one.** A shell can compute the answer server-side and return a single integer. A tool catalogue without an aggregate operation must list the directory, then list each subdirectory, and count by reasoning over every listing it has pulled into context. The task therefore contains one item where the two surfaces are structurally unequal, by design, because that inequality is the thing under study.

3.2 Arms

Each scaffolding/model pair runs twice:

- **MCP arm** — the official github/github-mcp-server (v1.8.0) attached over stdio in its default toolset configuration.
- **CLI arm** — the scaffolding’s native shell, with the authenticated gh CLI available on PATH and no MCP server attached.

Every scaffolding is run **in its default configuration**. We do not prune tool registries, trim built-ins or tune prompts. The question is what a practitioner gets when they connect a server and start working, not what a tuned harness can be made to do.

Thinking or extended reasoning is disabled, or set to the lowest available setting, on every scaffolding. Reasoning tokens are a large and separately-varying cost that would swamp the effect under study.

3.3 Matrix

Scaffolding	Models	MCP arm	CLI arm
Claude Code	sonnet-5, opus-5, fable-5	✓	✓
Codex	gpt-5.6 sol/luna/terra, glm-5.2, qwen3.5:122b, qwen3.6:27b	✓	✓
qwen-code	same six	✓	✓
pi	same six	🚫 void	✓

Two cells in this table are structural rather than incidental. **pi ships no MCP client** — no flag, no configuration key, no mention in its help output — so its MCP cells are reported void. A void cell is not a zero: “this scaffolding cannot do MCP” and “MCP costs this scaffolding nothing” are opposite claims, and a table that renders both as an empty cell asserts the wrong one. **Claude Code reaches only Anthropic models**, having no custom-endpoint configuration, so it is not run against the open-weights models.

3.4 Measurements

Per run: initial (first-request) prompt tokens; total input and output tokens; cached tokens and cache-hit rate; tool-call count; **the register of which tools were called**; tool-call success ratio; task completion percentage; theoretical cost; and wall-clock time.

The four scaffoldings report telemetry in four incompatible formats, three of them undocumented. We established each by running the scaffolding and inspecting its output rather than by reading its documentation, and the adapters record a field as *absent* rather than substituting zero when a scaffolding does not report it — a missing cache figure and a genuine zero support opposite conclusions.

3.5 Cost model

Cost is **theoretical**, computed by applying OpenRouter’s public per-token list prices uniformly to every cell, including cells served on local hardware.

This is deliberate. Half these models run on a machine in the room, where marginal cost is electricity, and the rest bill through different arrangements — a subscription, a metered key. Comparing invoices would compare accounting arrangements rather than workloads. One public, dated price list applied uniformly gives a figure that is comparable across the matrix and reproducible by anyone.

Cached input is billed as input, with no discount assumed, because providers discount cached reads at different rates and local inference has no billing concept at all. Cache-hit rate is reported as its own dimension, so a reader who wants to apply a particular vendor’s discount can.

Real money spent is recorded separately and never folded into this figure.

3.6 What wall-clock time can and cannot say

We record wall time and we do not compare it across the matrix. Locally-served models run on one machine with one request in flight; hosted models run on provider infrastructure with unknown batching. A timing comparison between them measures the hardware, not the surface. Within a single model, timings are also sensitive to cache state (§4.4). We publish the numbers because they are cheap to record and occasionally diagnostic, and we draw no conclusions from them.

4. Measurement hazards

Every hazard in this section was found by getting a wrong answer first. We report them because each produces *plausible* numbers, which is the dangerous kind of wrong, and because a replication that does not control them will not reproduce our results.

4.1 An MCP arm that silently is not one

Two of the four scaffoldings started with **no MCP tools attached and reported no error**. qwen-code leaves an unapproved server in a pending state and runs anyway; Codex ignored an inline configuration override. In both cases the agent fell back to its shell, completed the task, and produced a clean set of numbers that would have been recorded as an MCP measurement.

The harness therefore asserts, after every MCP run, that at least one MCP tool was actually called, and voids the cell otherwise.

4.2 A scaffolding reporting success on a failed task

In one run the scaffolding’s own result event reported subtype: "success" and `is_error: false` for an answer that was incomplete — the agent was still counting files when it stopped. A harness that trusts the scaffolding’s status would score it as a pass.

Completion is therefore scored only by the rubric, against ground truth, and never from the scaffolding’s self-report.

4.3 Configuration that silently changes what is in context

Two settings in one scaffolding change the tool surface without changing anything the operator would think to record:

- a **tool-search budget** expressed as a percentage of the context window, which switches between declaring tool schemas upfront and fetching them on demand depending on whether they fit — so the same server produces different context costs at different registry sizes;

- an **approval mode**, which in a headless session does not merely block risky tools but **never registers them**, removing them from the model’s context entirely. At the default setting the shell tool is absent, and a CLI arm cannot exist.

Both must be pinned and reported. Neither is something a casual replication would think to state.

4.4 Cache state that leaks between runs

The local inference server persists its KV cache to disk across runs. Whichever arm runs after a similar one therefore starts warm and appears dramatically cheaper and faster. In an early sweep the fastest run of the set was fastest only because of what had run before it.

Runs are interleaved by arm and the local cache is cleared between cells.

4.5 Reasoning that does not terminate

With extended thinking enabled, the locally-served GLM model has produced 26,000+ reasoning tokens without converging on a task it solves in ~500 tokens with thinking disabled. Reasoning is disabled server-side, and every run is checked for reasoning output and for an implausible output-token count.

5. Results

⟨TBD — generated by bench/analyze.py.⟩

5.1 Results table

⟨TBD⟩

5.2 Token cost

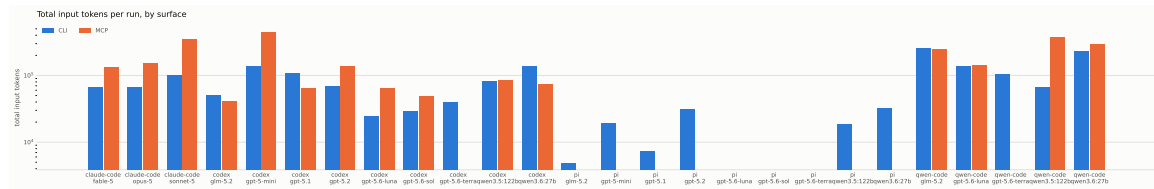


Figure 1: Total input tokens per run, by surface

⟨TBD⟩

5.3 Tool calls and the tool register

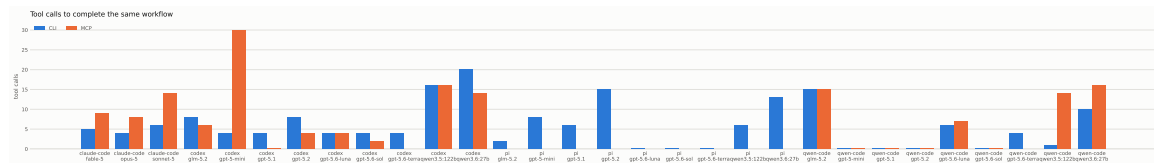


Figure 2: Tool calls to complete the same workflow

⟨TBD⟩

5.4 Task completion

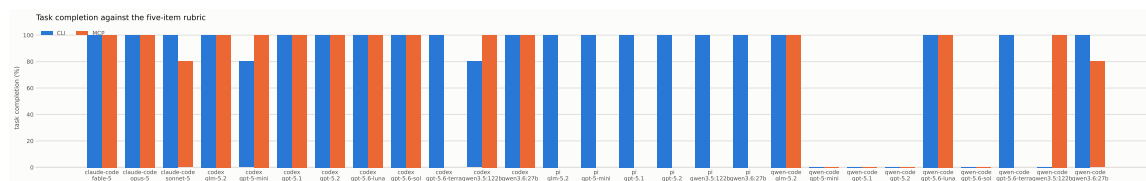


Figure 3: Task completion against the five-item rubric

<TBD>

5.5 Cost

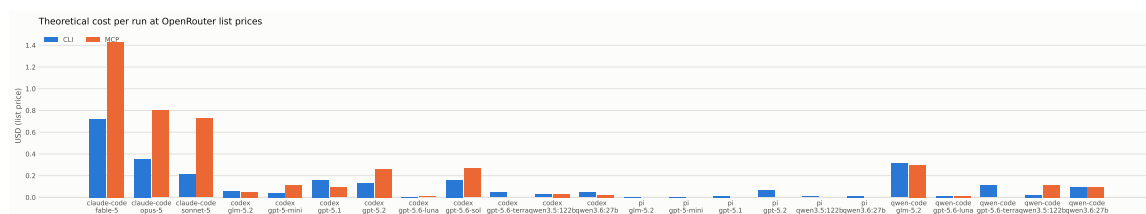


Figure 4: Theoretical cost per run at list prices

〈TBD〉

6. Discussion

<TBD>

7. Limitations

One run per cell. The matrix is broad and shallow. A single run per cell establishes that an effect is reproducible in principle, not how often it occurs, and no variance is reported. Deepening the matrix is the obvious next version and the harness supports it directly.

One task, in one domain. The workflow is a GitHub task, and the discriminating item rewards a surface that can aggregate server-side. A different domain — one where the tool catalogue offers exactly the operation needed and the shell does not — would plausibly move the result the other way. We publish the task so that claim can be tested rather than argued.

Default configuration is a choice, not a neutral baseline. Running every scaffolding as it ships measures what a practitioner encounters, and it also means a scaffolding with a bloated default tool set is penalised for a decision that a tuned deployment would reverse.

Prices move. The cost column is a snapshot of one public price list on one date, and it is a modelled figure rather than a billed one.

We wrote the CLI-side affordance. Where a skill document is supplied to the CLI arm, we wrote it, and a reviewer is right to ask whether it is a fair match for the tool catalogue it is compared against. It is published in full for exactly that reason.

8. Conclusion

<TBD>

Data and code availability

Harness, task, rubric, adapters, cost model, fixture pointer and every raw run log are at github.com/Lamb-Project/mcp-vs-cli-bench, archived to Zenodo.

Reproducing the numbers requires four conditions that change results if skipped, each documented in the repository README: reasoning disabled server-side; the fixture pinned at its tag; the pinned fastapi version if the LiteLLM proxy is used; and the local KV cache cleared between runs.

References

⟨TBD⟩